

Timsove ulohy Documentacia

Uloha 1 (T1A) : Analyticke riesenie LDR:

Zvolte si z Tutorialu LDR ($n=2$ alebo $n=3$) s pravou stranou a **vyrieste ju v casovej oblasti a pomocou LT. (odorucanie: zvolte si jednoduchu funkciu $u(t)$ ako pravu stranu pre LDR).** Sucastou Vasho riesenia by malo byt **porovnanie vysledkov** ziskanych z **riesenia LDR s vyuzitim LT s riesenim ziskanim v casovej oblasti.**

Main.m

```
%% Riešenie ODR pomocou metódy Laplaceovej transformácie a porovnanie s dsolve
% Deklarácia symbolických premenných a funkcie
syms y(t) s t
assume(t, 'real'); % Určujeme, že t je reálne číslo

%% 1. Riešenie pomocou Laplaceovej transformácie
% Definujeme Laplaceovu transformáciu pravej strany:  $L\{\cos(2t)\} = s / (s^2 + 2^2)$ 
U_s = s / (s^2 + 4);

% Laplaceova transformácia ľavej strany s zohľadnením počiatočných podmienok:
%  $L\{y''\} = s^2 Y(s) - s y(0) - y'(0)$ 
%  $L\{4y\} = 4 Y(s)$ 
% Rovnica v s-doméne:  $(s^2 Y(s) - s y(0) - y'(0)) + 4 Y(s) = U_s$ 
% Dosadíme počiatočné podmienky  $y(0)=1, y'(0)=0$ :
%  $(s^2 Y(s) - s*1 - 0) + 4 Y(s) = s / (s^2 + 4)$ 
%  $s^2 Y(s) - s + 4 Y(s) = s / (s^2 + 4)$ 
%  $Y(s) * (s^2 + 4) = s + s / (s^2 + 4)$ 
%  $Y(s) * (s^2 + 4) = (s * (s^2 + 4) + s) / (s^2 + 4)$ 
%  $Y(s) * (s^2 + 4) = (s^3 + 4s + s) / (s^2 + 4)$ 
%  $Y(s) = (s^3 + 5s) / ((s^2 + 4) * (s^2 + 4))$ 
%  $Y(s) = (s^3 + 5s) / (s^2 + 4)^2$ 

Y = (s^3 + 5*s) / (s^2 + 4)^2;

% Rozklad na elementárne zlomky (môže byť zložitý pre opakujúce sa korene)
% Y_part = partfrac(Y, s); % Nemusí poskytnúť jednoduchý tvar pre tento prípad
% disp('Partial expression for Y(s):');
% pretty(Y) % Zobrazí Y(s)

% Inverzná Laplaceova transformácia na nájdenie y(t)
yLaplace = ilaplace(Y, s, t);
yLaplace = simplify(yLaplace);

disp('Riešenie získané inverznou Laplaceovou transformáciou:');
pretty(yLaplace)

%% 2. Riešenie pomocou dsolve (priame riešenie v časovej doméne)
% Definícia diferenciálnej rovnice a počiatočných podmienok
Dy = diff(y,t);
ode = diff(y,t,2) + 4*y == cos(2*t); % Rovnica:  $y'' + 4y = \cos(2t)$ 
conds = [y(0)==1, Dy(0)==0]; % Počiatočné podmienky:  $y(0) = 1, y'(0) = 0$ 

% Riešenie pomocou dsolve
y_dsolve = dsolve(ode, conds);
y_dsolve = simplify(y_dsolve);

disp('Riešenie získané pomocou dsolve:');
pretty(y_dsolve)

%% 3. Porovnanie oboch riešení
% Kontrola, či sú obe riešenia ekvivalentné
if simplify(yLaplace - y_dsolve) == 0
    disp('Obe riešenia sú identické.');
```

```

    disp('Boli nájdené nezrovnalosti medzi riešeniami.');
```

end

```

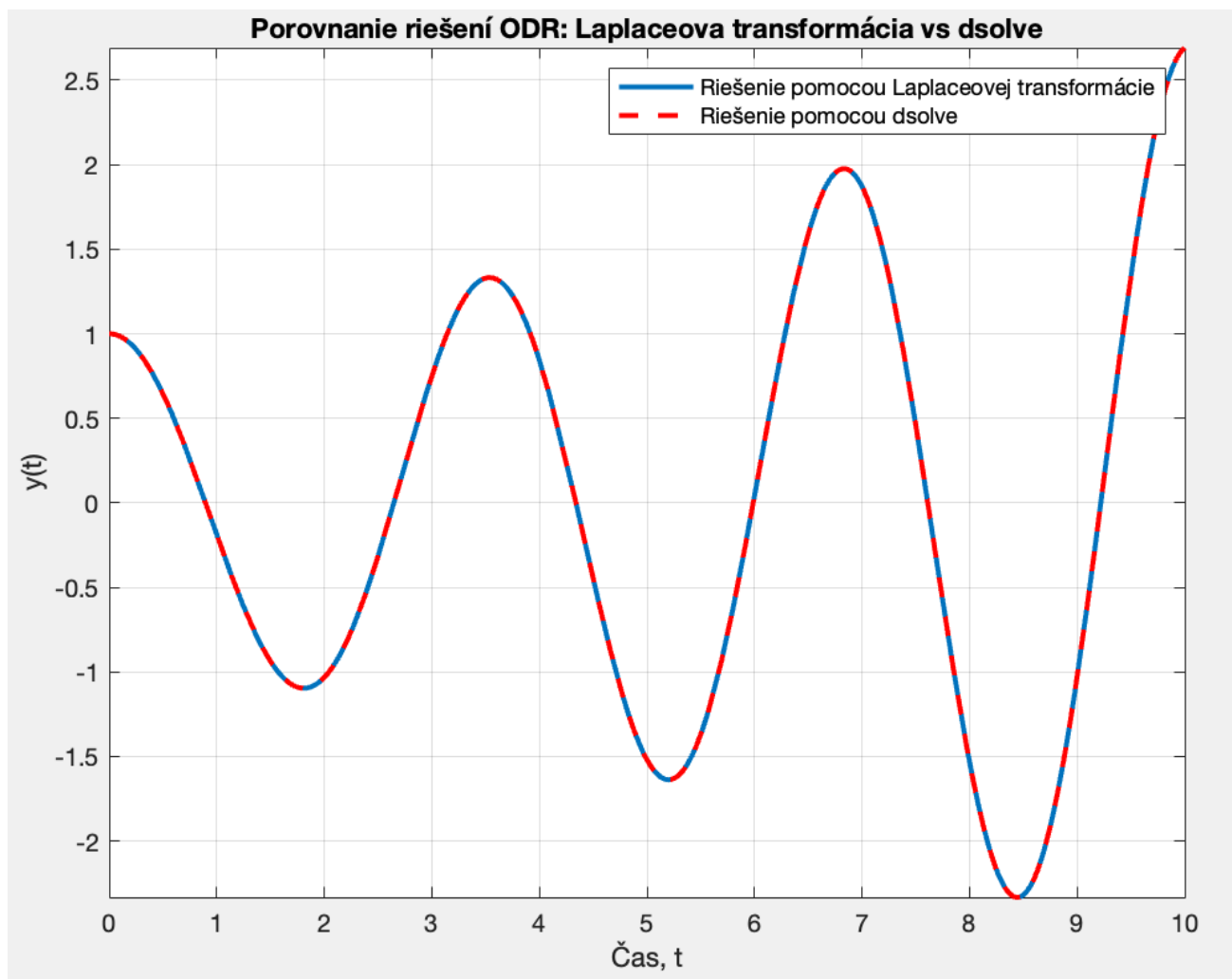
%% 4. Vykreslenie grafov oboch riešení pre vizuálne porovnanie
figure("Name", "Porovnanie riešení")
fplot(yLaplace, [0, 10], 'LineWidth', 2); % Vykreslenie grafu Laplaceovho
riešenia
hold on
fplot(y_dsolve, [0, 10], '--r', 'LineWidth', 2); % Vykreslenie grafu dsolve
riešenia
xlabel('Čas, t');
ylabel('y(t)');
title('Porovnanie riešení ODR: Laplaceova transformácia vs dsolve');
legend('Riešenie pomocou Laplaceovej transformácie', 'Riešenie pomocou dsolve');
grid on
hold off
```

Riešenie získané inverznou Laplaceovou transformáciou:

$$\cos(2t) + \frac{t \sin(2t)}{4}$$

Riešenie získané pomocou dsolve:

$$\cos(2t) + \frac{t \sin(2t)}{4}$$



Ulohy T1_B - Numericke riesenie lineárnych diferencialnych rovníc (LDR)

Riesenie LDR s vyuzitim METOD NUMERICKEJ MATEMATIKY a solverov simulacneho prostredia jazyka MATLAB/Simulink.

V ulohach na ziskanie **NUMERICKEHO RIESENIA LDR** s vyuzitim **riesicov jazyka MATLAB** je **nutne**:

- a) prepisat LDR do substitucneho kanonickeho tvaru (SKT),**
- b) vyriesit LDR v SKT s vyuzitim ODE riesica (funkcie jazyka MATLAB, vlastne funkcie) v zmysle zadania ulohy,**
- c) vykreslit riesenie a jeho derivacie v popisanych grafoch** (nazov grafu, oznacene osi v grafe, uvedenie legendy s popisom).

Uloha 1(T1_B):

Analytickym vypoctom vyriesene LDR (2./3.radu) s konstantnymi koeficientmi z Tutorialu (<http://matlab.fei.tuke.sk/zar/subory/cvicenia/Cvicenie1.pdf>) **vyrieste ODE riesicom/soverom (ode23, ode45) jazyka Matlab.**

Simulaciou overte zhodu vo vysledkoch, ktore boli ziskane **analytickym riesenim LDR** s vysledkami ziskanymi **pouzitim ODE riesicov.**

main.m

```
%% T1_B: Riešenie ODR druhého rádu pomocou MATLABovských solverov a vlastnej
metódy RK4
% Vyčistenie pracovného priestoru
clear;
clc;

%% Definícia parametrov systému
% Systém:  $y''(t) + 4y(t) = \cos(2t)$ 
% Prepíšeme do kanonickej formy substitúcie (stavový priestor)
%  $x_1 = y, x_2 = y'$ 
% Potom:
%  $x_1' = x_2$ 
%  $x_2' = -4x_1 + \cos(2t)$ 
% Definícia času simulácie
tspan = [0 10]; % Zväčšený rozsah pre zobrazenie oscilácií
% Počiatočné podmienky:  $y(0) = 1, y'(0) = 0$ 
x0 = [1; 0];

%% Riešenie pomocou ode45
% ode45 – je univerzálny solver pre nestacionárne ODR
[t_ode45, x_ode45] = ode45(@systemDynamics, tspan, x0);

%% Riešenie pomocou ode23
% ode23 – je jednoduchší solver, vhodný pre systémy, ktoré nie sú tuhé
[t_ode23, x_ode23] = ode23(@systemDynamics, tspan, x0);

%% Riešenie pomocou ode15s (tuhý solver, pre kontrolu robustnosti)
% ode15s – je určený pre tuhé systémy, ale môže fungovať aj s tými, ktoré nie sú
tuhé
[t_ode15s, x_ode15s] = ode15s(@systemDynamics, tspan, x0);

%% Riešenie pomocou vlastnej metódy RK4
dt = 0.01; % Časový krok pre RK4
% rk4Solver – je vlastná implementácia metódy Runge-Kutta 4. rádu
[t_rk4, x_rk4] = rk4Solver(@systemDynamics, tspan, x0, dt);

%% Vykreslenie grafov riešení ( $x_1 = y(t)$ )
figure;
plot(t_ode45, x_ode45(:,1), 'b-', 'LineWidth', 2); hold on;
plot(t_ode23, x_ode23(:,1), 'r--', 'LineWidth', 2);
plot(t_ode15s, x_ode15s(:,1), 'g-.', 'LineWidth', 2);
plot(t_rk4, x_rk4(:,1), 'k:', 'LineWidth', 2);
xlabel('Čas t [s]');
ylabel('Stav  $x_1(t) = y(t)$ ');
title('Porovnanie riešení pre  $y(t)$ ');
legend('ode45', 'ode23', 'ode15s', 'RK4 (vlastné)', 'Location', 'best');
grid on;
hold off; % Dôležité po dokončení pridávania grafov na aktuálnu os

%% Vykreslenie grafov derivácií ( $x_2 = dy/dt$ )
figure;
plot(t_ode45, x_ode45(:,2), 'b-', 'LineWidth', 2); hold on;
plot(t_ode23, x_ode23(:,2), 'r--', 'LineWidth', 2);
plot(t_ode15s, x_ode15s(:,2), 'g-.', 'LineWidth', 2);
```

```

plot(t_rk4, x_rk4(:,2), 'k:', 'LineWidth', 2);
xlabel('Čas t [s]');
ylabel('Stav x_2(t) = dy/dt');
title('Porovnanie riešení pre dy/dt');
legend('ode45', 'ode23', 'ode15s', 'RK4 (vlastné)', 'Location', 'best');
grid on;
hold off; % Dôležité po dokončení pridávania grafov na aktuálnu os

```

rk4Solver.m

```

function [t, x] = rk4Solver(dynamics, tspan, x0, dt)
    t = tspan(1):dt:tspan(2); % time vector
    N = length(t); % number of steps
    n = length(x0); % state dimension
    x = zeros(N, n); % preallocate state matrix
    x(1,:) = x0'; % set initial condition

    for i = 1:N-1
        k1 = dynamics(t(i), x(i,:));
        k2 = dynamics(t(i) + dt/2, x(i,:) + dt/2 * k1);
        k3 = dynamics(t(i) + dt/2, x(i,:) + dt/2 * k2);
        k4 = dynamics(t(i) + dt, x(i,:) + dt * k3);

        x(i+1,:) = x(i,:) + (dt/6)*(k1' + 2*k2' + 2*k3' + k4');
    end
end

```

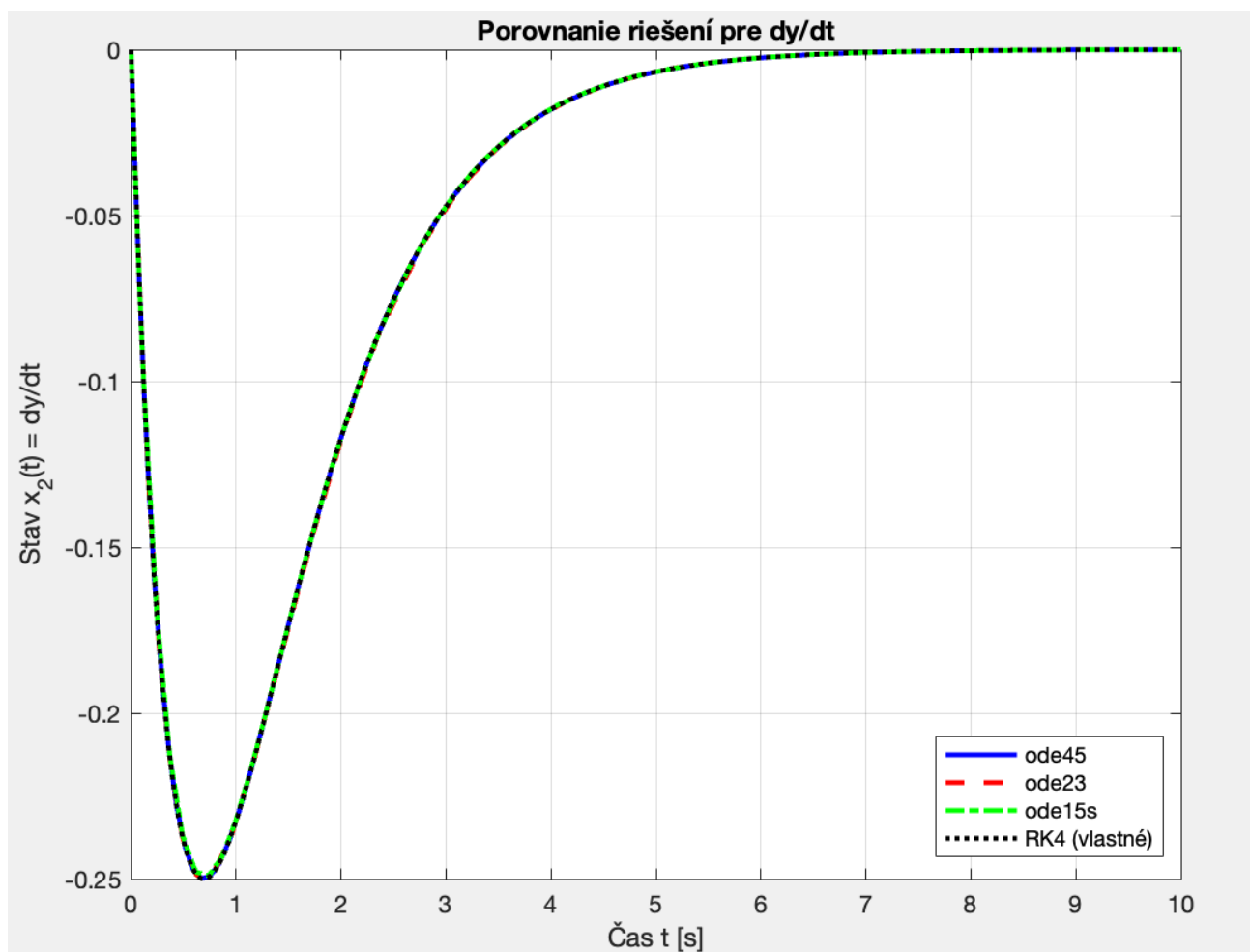
systemDynamics.m

```

function dx = systemDynamics(t, x)
    % System dynamics for canonical form of ODE
    % x(1) = y
    % x(2) = dy/dt

    dx = zeros(2,1);
    dx(1) = x(2);
    dx(2) = -3*x(2) - 2*x(1) + 1; % Constant input u(t) = 1
end

```

Ulohy T2_A - Modelovanie a simulácia LDS v prostredí MATLAB

V riešení **ULOHY (T2_A)** uvažujte **lineárne dynamické systémy popísané LDR** s konštantnými koeficientmi (**rieste jeden z uvedených LDS**)

- **elektrický systém 2. radu - RLC obvod** (konfigurácia pre nabíjanie kondenzátora alebo vybíjanie kondenzátora, sledované veličiny sú: prúd a úbytky napätí)
- **elektromechanický systém - DC motor** (sledované veličiny: otáčky a poloha)

Ulohy(T2_A): Jeden z uvedených dynamických systémov analyzujte z pohľadu **vytvorenia SIMULACNEHO modelu**:

- uvedte **vstupy/vystupy LDS + parametre systému aj s jednotkami**,
- odvodte a zapíšte diferenciálne rovnice tvoriace **matematický model systému**;
- v prípade potreby rozpíšte **ZISKANÝ MATEMATICKÝ MODEL** do **substitučného kanonického tvaru (SKT)**,
- simulujte v prostredí MATLAB** pomocou **funkcie ode45** alebo iného vhodného **ODE riešiča pri zvolení vhodných vstupných signalov** (napr. **zvoľte ako vstup $u(t)$** - jednotkový skok, harmonickú funkciu, rýchlostnú funkciu, atď)
- vykreslite priebehy všetkých sledovaných veličín v case**, ktoré vhodne rozmiestnite do grafov s odpovedajúcim názvom grafu, označením osí, legendou (*farebne rozlíšiť viaceré funkcie v grafe*)

Pozn. Ak Vase riešenie bude **obsahovať simulacné modely oboch systémov**, tak realizujte riešenie napr. **v rámci jedného m-filu rozčleneného do sekcií**, v rámci ktorých opakovanne voláte **funkciu ode45** vždy s iným systémom, ktorého numerické riešenie je výsledkom simulácie LDS na vhodný vstupný signál.

main.m

```
%% T2_A: Simulácia RLC obvodu (nabíjanie kondenzátora)
% Vyčistenie pracovného priestoru
clear;
clc;

%% Parametre RLC obvodu (pretlmený prípad)
R = 10;          % Odpor [Ohm]
L = 0.1;         % Indukčnosť [H]
C = 0.01;        % Kapacita [F]

% Definícia času simulácie
tspan = [0 0.5]; % Zmenšený časový interval pre pretlmenú odozvu

% Počiatočné podmienky: napätie na kondenzátore a jeho derivácia (prúd)
x0 = [0; 0]; %  $x_1(0) = u_C(0) = 0V$ ,  $x_2(0) = u_C'(0) = i(0)/C = 0 A/F$ 

%% Riešenie pomocou ode45
% Používame anonymnú funkciu na odovzdanie parametrov R, L, C do rlcSystem
[t, x] = ode45(@(t, x) rlcSystem(t, x, R, L, C), tspan, x0);

% Vstupný signál (jednotkový skok) – pre vykreslenie grafu
u = ones(size(t)); %  $u(t) = 1$  pre  $t \geq 0$ 

%% Vykreslenie grafov napätia na kondenzátore ( $u_C$ ) a prúdu ( $i$ )
figure;
plot(t, x(:,1), 'b-', 'LineWidth', 2); hold on; %  $x(:,1)$  – je  $u_C(t)$ 
plot(t, C*x(:,2), 'r--', 'LineWidth', 2); %  $i(t) = C * u_C'(t)$ , kde  $u_C'(t)$  –
je  $x(:,2)$ 
xlabel('Čas t [s]');
ylabel('Amplitúda');
title('Odozva RLC obvodu (nabíjanie kondenzátora) – Pretlmený prípad');
legend('Napätie na kondenzátore  $u_C(t)$ ', 'Prúd  $i(t)$ ', 'Location', 'best');
grid on;
hold off;

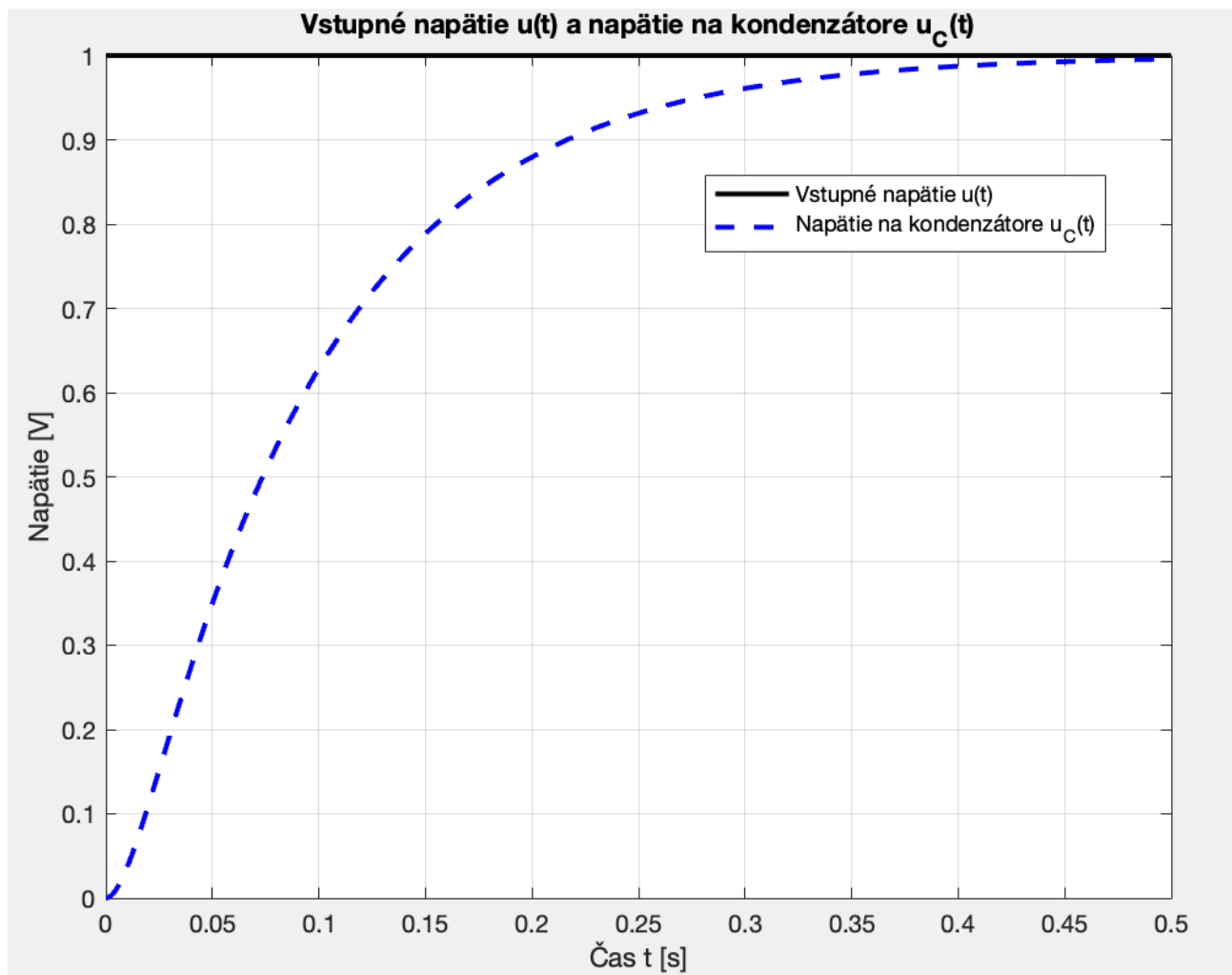
%% Vykreslenie grafov vstupného napätia a napätia na kondenzátore
figure;
plot(t, u, 'k-', 'LineWidth', 2); hold on;
plot(t, x(:,1), 'b--', 'LineWidth', 2); %  $x(:,1)$  – je  $u_C(t)$ 
xlabel('Čas t [s]');
ylabel('Napätie [V]');
title('Vstupné napätie  $u(t)$  a napätie na kondenzátore  $u_C(t)$ ');
legend('Vstupné napätie  $u(t)$ ', 'Napätie na kondenzátore  $u_C(t)$ ', 'Location',
'best');
grid on;
hold off;
```

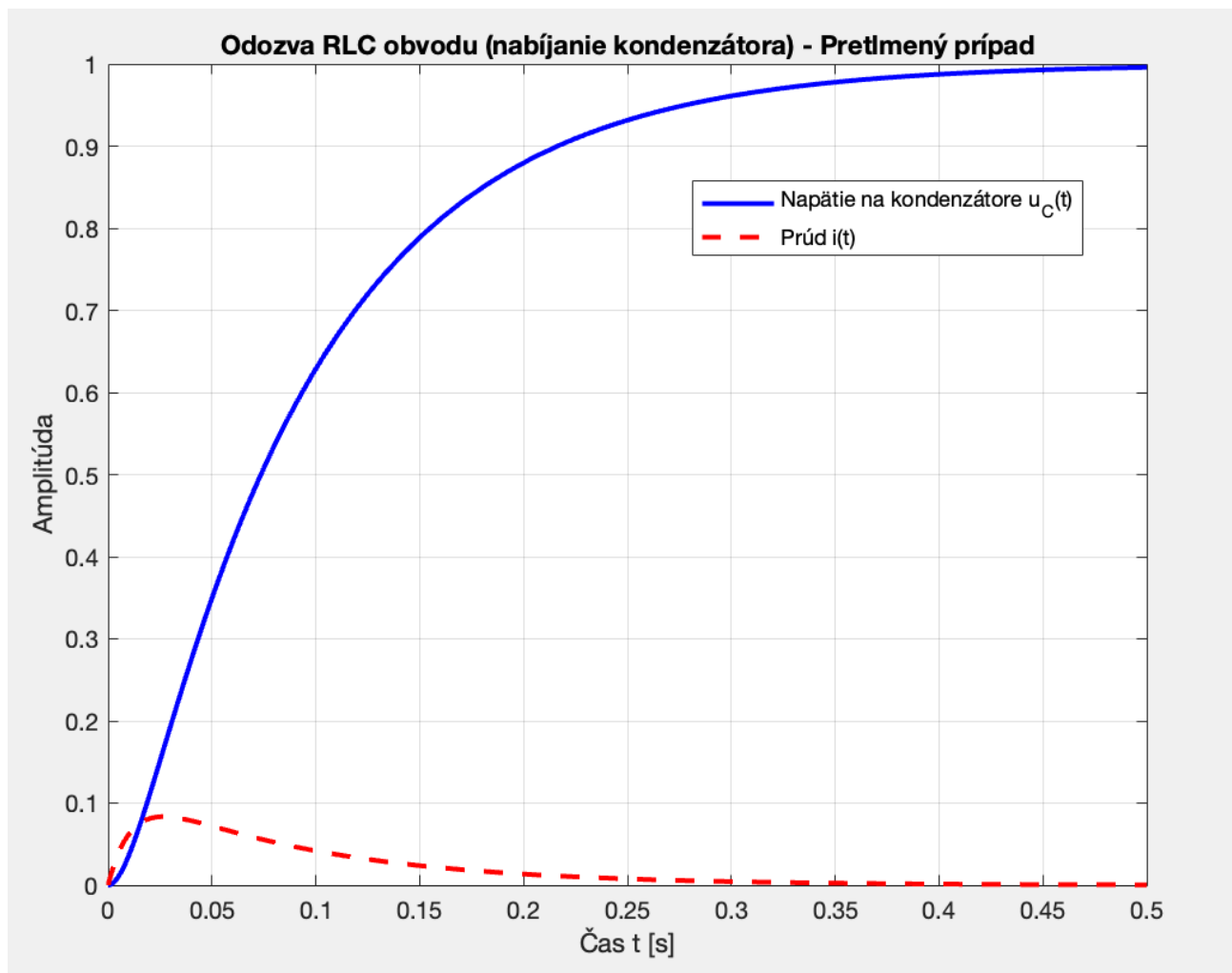
rlcSystem.m

```
function dx = rlcSystem(t, x, R, L, C)
    % System dynamics for RLC circuit in substitution canonical form
    % x(1) = uC(t) - capacitor voltage
    % x(2) = duC/dt

    % Input signal: unit step
    u = 1;

    dx = zeros(2,1);
    dx(1) = x(2);
    dx(2) = (1/(L*C))*(u - R*C*x(2) - x(1));
end
```





Ulohy T2_B - Modelovanie a simulacia NELINEARNÝCH DS v prostredí MATLAB

Modelovanie dynamických systémov popísaných "NELINEARNÝMI diferenciálnymi rovnicami" v prostredí MATLAB

Uvážujte dynamické systémy popísané **nelineárnymi diferenciálnymi rovnicami** (NDR):

- **van der Polov oscilátor** - https://en.wikipedia.org/wiki/Van_der_Pol_oscillator
- **matematické kyvadlo** - https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-832-underactuated-robotics-spring-2009/readings/MIT6_832s09_read_ch02.pdf

Uloha 1 (T2_B)

Kazdy z uvedených **nelineárných dynamických systémov** analyzujte z pohľadu:

- 1a) **vytvorenia simulacného modelu** (uvedte vstupy/vystupy/parametre),
- 1b) **zapíšte diferenciálne rovnice tvoriace matematický model systému** (matematický model je výsledkom analytickej identifikácie),
- 1c) odvodený model NDS **rozpíšte do substitucného kanonického tvaru**,
- 1d) implementujte matematický model NDS reprezentovaný NDR do prostredia MATLAB a **vykonajte simulácie** s využitím **funkcie ode45** alebo iného vhodného riešiča,
- 1e) **vykreslite priebehy všetkých sledovaných veličín v case**, vhodne ich rozmiestnite **do grafov s odpovedajúcim názvom, označením osí, legendou a mriežkou**.

Uloha2 (T2_B):

Programový výstup realizujte napr. **v rámci jedného m-filu rozčleneného do sekcií**, v rámci ktorých opakovane voláte funkciu **ode45** vždy s iným systémom.

main.m

```
%% Simulácia Van der Pol oscilátora a Matematického kyvadla
% Upozornenie: Funkcie van_der_pol a pendulum sú definované ako lokálne
% funkcie na konci tohto súboru.

%% Simulácia Van der Pol oscilátora
% Parametre
mu = 5; % Nelinearita oscilátora (zmenené z 1 na 5)

% Počiatočné podmienky
x0 = [2; 0]; % [x(0); x_dot(0)]

% Časový rozsah
tspan = [0 20];

% Riešenie ODR pomocou ode45
[t_vdp, x_vdp] = ode45(@(t, x) van_der_pol(t, x, mu), tspan, x0);

% Vykreslenie výsledkov
figure;
subplot(2,1,1);
plot(t_vdp, x_vdp(:,1));
title('Van der Pol Oscilátor - x(t)'); % Slovenský názov
xlabel('Čas [s]'); % Slovenský popis osi
ylabel('x');
grid on;

subplot(2,1,2);
plot(t_vdp, x_vdp(:,2));
title('Van der Pol Oscilátor - x''(t)'); % Slovenský názov
xlabel('Čas [s]'); % Slovenský popis osi
ylabel('x''');
grid on;

%% Simulácia Matematického kyvadla
% Parametre
g = 9.81; % gravitačné zrýchlenie [m/s^2]
l = 1.0; % dĺžka kyvadla [m]

% Počiatočné podmienky
theta0 = [pi/2; 2]; % [theta(0); theta_dot(0)] (zmenené z [pi/3; 0])

% Časový rozsah
tspan = [0 10];

% Riešenie ODR pomocou ode45
[t_pendulum, theta_pendulum] = ode45(@(t, theta) pendulum(t, theta, g, l),
tspan, theta0);

% Vykreslenie výsledkov
figure;
subplot(2,1,1);
plot(t_pendulum, theta_pendulum(:,1));
title('Matematické kyvadlo - \theta(t)'); % Slovenský názov
```



```

xlabel('Čas [s]'); % Slovenský popis osi
ylabel('\theta [rad]');
grid on;

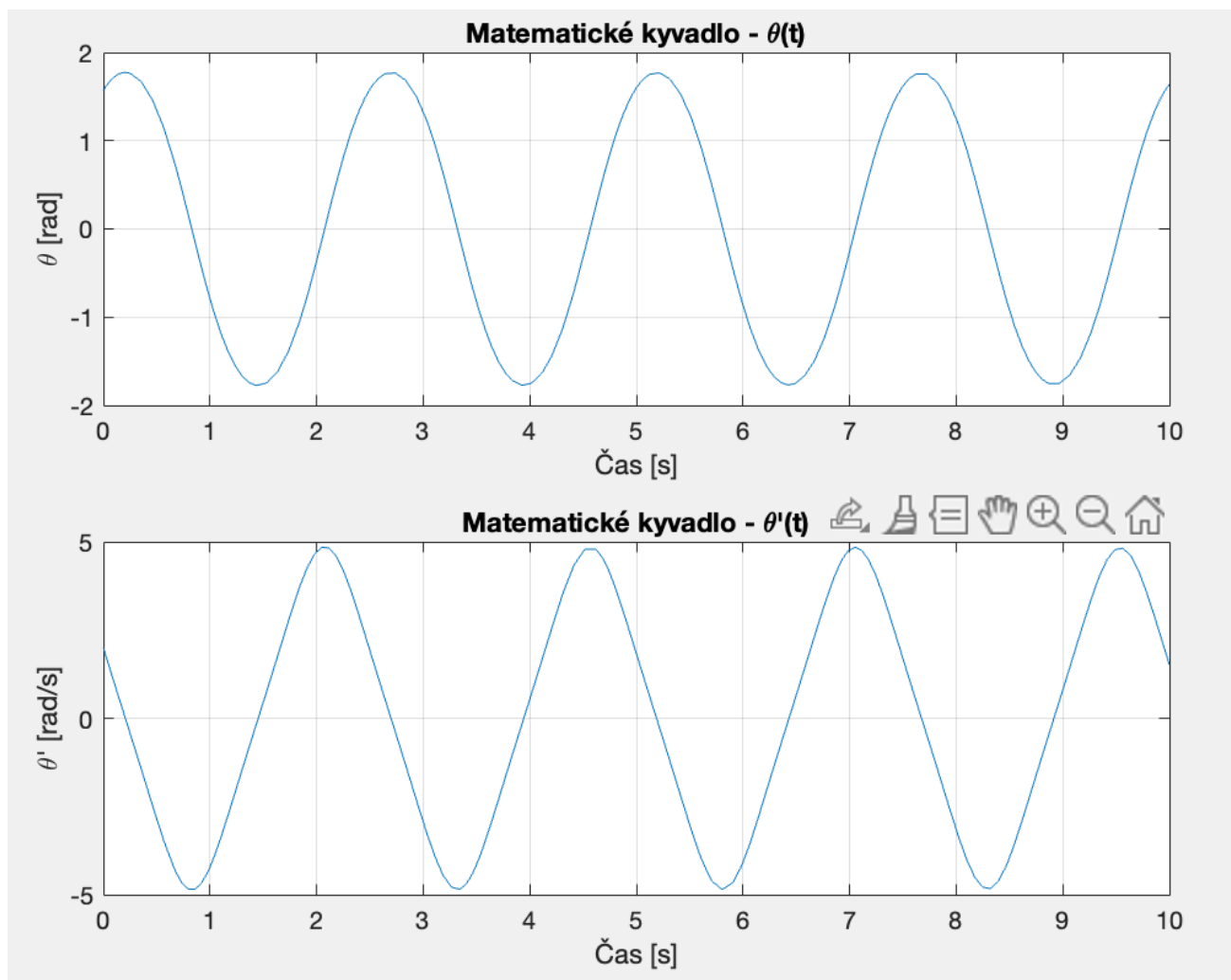
subplot(2,1,2);
plot(t_pendulum, theta_pendulum(:,2));
title('Matematické kyvadlo - \theta''(t)'); % Slovenský názov
xlabel('Čas [s]'); % Slovenský popis osi
ylabel('\theta'' [rad/s]');
grid on;

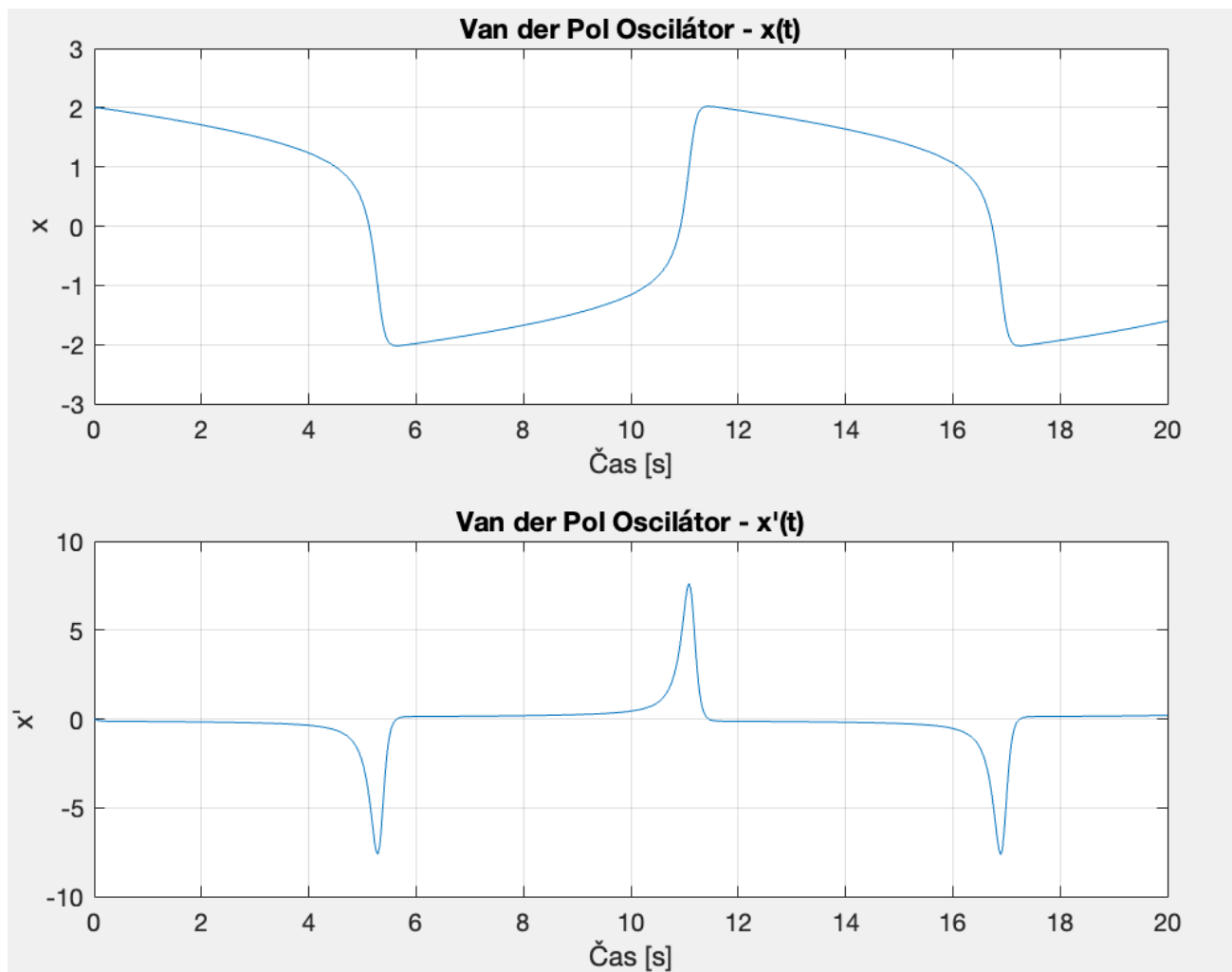
%% -----
% LOKÁLNE FUNKCIE (definované na konci hlavného skriptu)
%% -----

%% Funkcia pre Van der Pol oscilátor
% VAN_DER_POL: Definuje dynamiku Van der Pol oscilátora v stavovom priestore.
% x' = [x1'; x2'] = [x2; mu*(1-x1^2)*x2 - x1]
% Vstup: t - čas, x - vektor stavov [x1; x2], mu - parameter
% Výstup: dxdt - vektor derivácií [x1'; x2']
function dxdt = van_der_pol(~, x, mu)
    dxdt = zeros(2,1);
    dxdt(1) = x(2);
    dxdt(2) = mu*(1 - x(1)^2)*x(2) - x(1);
end

%% Funkcia pre Matematické kyvadlo
% PENDULUM: Definuje dynamiku matematického kyvadla v stavovom priestore.
% theta' = [theta1'; theta2'] = [theta2; -(g/l)*sin(theta1)]
% Vstup: t - čas, theta - vektor stavov [theta1; theta2], g - gravitácia, l -
dĺžka
% Výstup: dthetadt - vektor derivácií [theta1'; theta2']
function dthetadt = pendulum(~, theta, g, l)
    dthetadt = zeros(2,1);
    dthetadt(1) = theta(2);
    dthetadt(2) = -(g/l)*sin(theta(1));
end

```





Ulohy T3_A - Numericke riesenie LDR v prostredi Simulink – metoda postupneho znizovania radu derivacie

Riesenie LDR s vyuzitim solverov simulacneho prostredia MATLAB/Simulink metodu postupneho znizovania radu derivacie

V nasledujucich ulohach na ziskanie **numerickeho riesenia LDR s konstatnymi koeficientami** s vyuzitim **riesicov jazyka MATLAB** a **riesicov v prostredi SIMULINK** je **NUTNE**:

a) prepisat LDR do substitucneho kanonickeho tvaru (SKT),

b) naprogramovat zvolenu LDR v SKT v prostredi SIMULINK s vyuzitim metody postupneho znizovania radu derivacie a zvolit vhodny **ODE solver/riesic** (vsimnite si moznost volby **konstatneho/premenliveho kroku** pre zvoleny riesic)

c) vykreslit riesenie a jeho derivacie v popisanych grafoch s vyuzitim prikazov 2D grafiky v prostredi MATLAB (*nazov grafu, oznacene osi v grafe, uvedenie legendy s popisom*).

Poznamka.: Ziskane **numericke riesenie LDR** si najprv vykreslite v prostredi Simulink (napr. pre rychlu kontrolu s vyuzitim bloku „**oscilloscope**“). Nasledne ziskane riesenie LDR v Simulinku ulozte **do vektorov v prostredi Workspace** a s vyuzitim funkcie „**plot**“ ich ako v predoslych ulohach (**T1 – T2**) znazornite v prostredi MATLAB.

Uloha 1(T3_A):

Naprogramujte zvolenu LDR ($n=2/n=3$) s konstatnymi koeficientami v prostredi Simulink.

Vyskusajte si **volbu ponukanych solverov a volbu integracneho kroku** (konstatny krok/premenlivy krok).

Ziskane numericke vysledky (riesenie LDR a jeho derivacie) znazornite najprv v ramci prostredia Simulink s vyuzitim bloku „**oscilloscope**“ a potom ich ulozte do vektorov, ktore vyuzijete vo funkcii „**plot**“ na vykreslenie vysledkov.

V tejto ulohu budete vyuzivat uz ziskane NUMERICKE riesenia z prostredia MATLAB, pripadne z analytickeho riesenia.

a) analyticky vypocet zvolenych LDR (2./3.radu) s konstantnymi koeficientmi z Tutorialu (<http://matlab.fei.tuke.sk/zar/subory/cvicenia/Cvicenie1.pdf>) a ich numericke riesenia **ODE riesicom** (**ode23, ode45**) v jazyku Matlab,

b) simulaciou overenu zhodu vo vysledkoch, ktore boli ziskane analytickym riesenim LDR s vysledkami ziskanyimi pouzitim ODE riesicov jazyka MATLAB .

main.m

```
figure;
plot(out.tout, out.y(:,2), 'DisplayName', 'y(t)');
hold on;
plot(out.tout, out.dy(:,2), '--', 'DisplayName', 'dy/dt');
title('Numerické riešenie LDR v Simulinku');
xlabel('čas [s]');
ylabel('hodnoty');
legend show;
grid on;

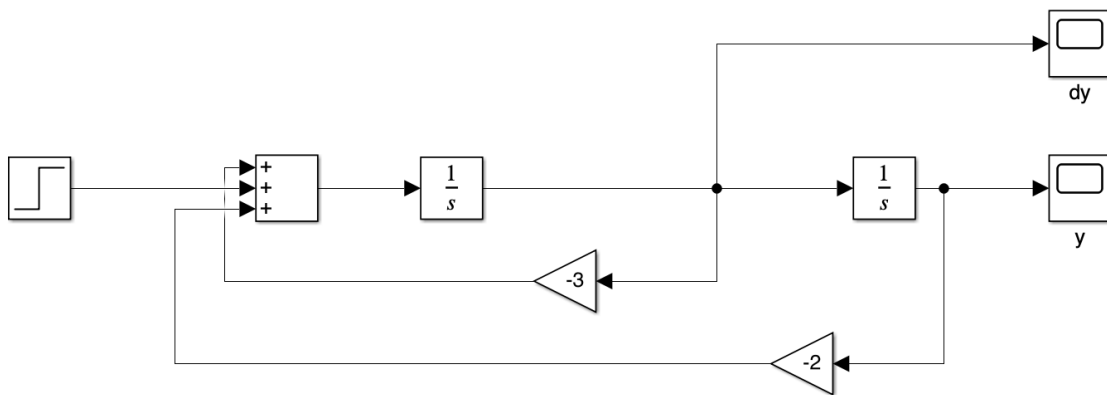
hold off;

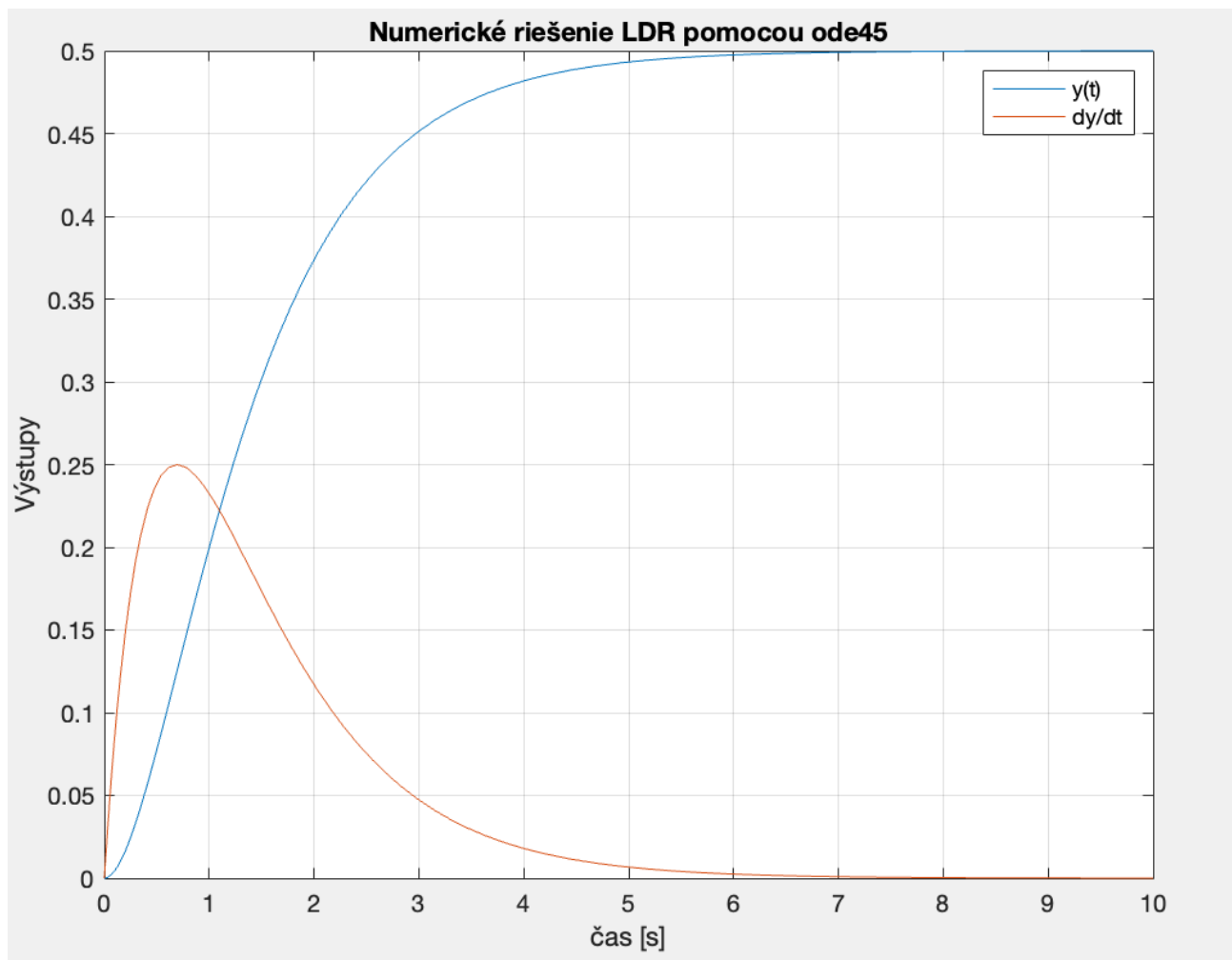
% Definícia rovnice
LDR = @(t, y) [y(2); -3*y(2) - 2*y(1) + 1]; % Jednotkový vstup

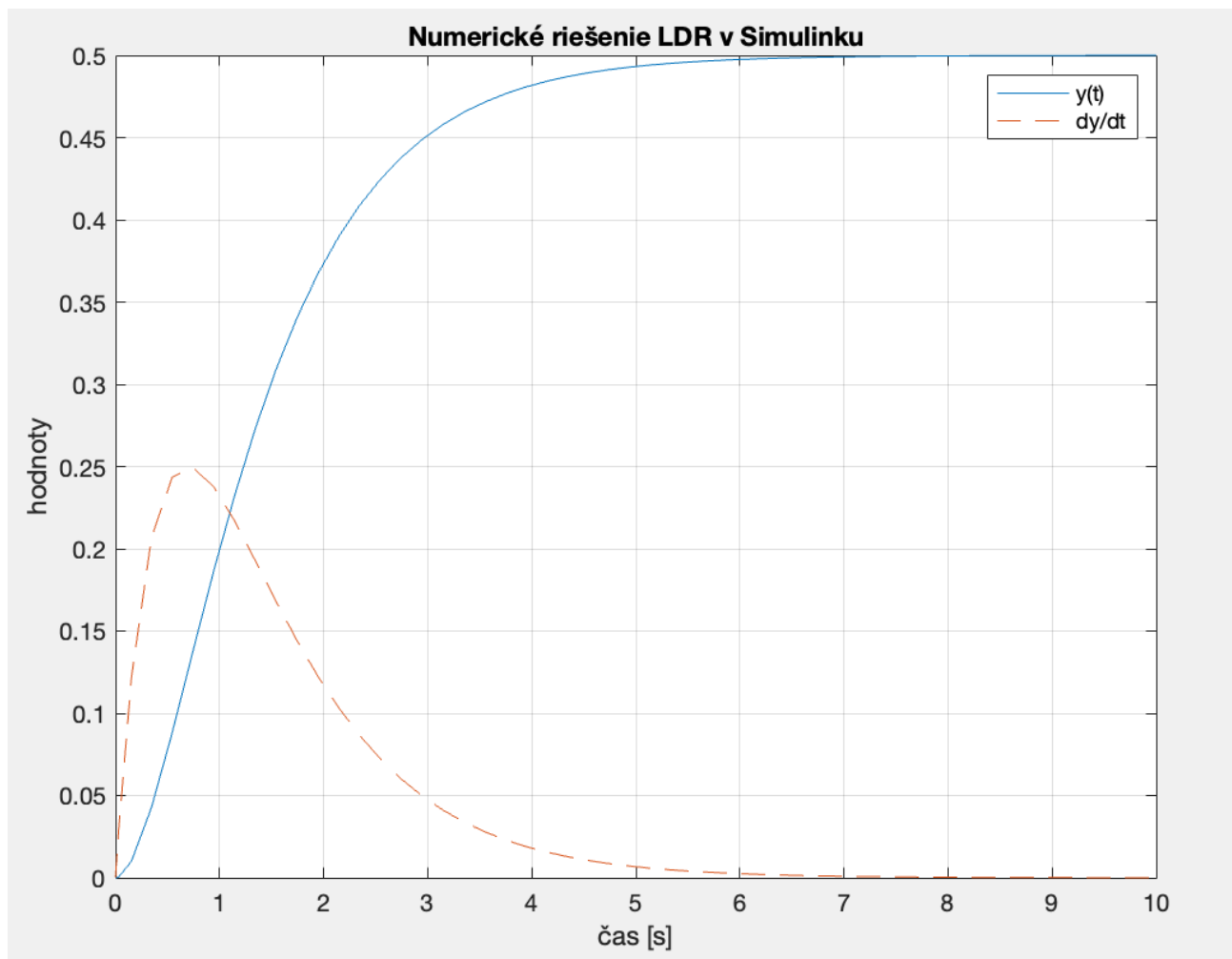
% Počiatočné podmienky
y0 = [0; 0];

% Simulácia
[t, y] = ode45(LDR, [0 10], y0);

% Plot
figure;
plot(t, y(:,1), 'DisplayName', 'y(t)');
hold on;
plot(t, y(:,2), 'DisplayName', 'dy/dt');
title('Numerické riešenie LDR pomocou ode45');
xlabel('čas [s]');
ylabel('Výstupy');
legend;
grid on;
```







Ulohy T3_B - Modelovanie a simulácia fyzikálnych LDS v prostredí Simulink

V riešení **ULOHY (T3_B)** uvažujte **dynamicke systémy popísané LDR** s konštantnými koeficientmi :

- **elektrický systém 2. radu - RLC obvod**(konfigurácia pre nabíjanie kondenzátora a vybíjanie kondenzátora, sledované veličiny: prúd a úbytky napätí)
- **elektromechanický systém - DC motor**(sledované veličiny: otáčky a poloha)

Ulohy(T3_B):

Kazdy z LDS naprogramujte a analyzujte z pohľadu **vytvorenia SIMULACNEHO modelu v Simulinku**:

- avedte **vstupy/vystupy s popisom parametrov fyzikálnych modelov**,
- zapíšte diferenciálne rovnice tvoriace **matematický model systému (PREVZAT ODVODENE MODELÝ z predoslych uloh alebo z Literatúry)**,
- ZISKANE MATEMATICKE MODELÝ** zapíšte do **substitucného kanonického tvaru (SKT)**
- naprogramujte zvolené modely LDS v prostredí Simulink** s využitím blokov a S-funkcií programového prostredia Simulink,
- numerické riešenie** nájdite pri výbere vhodného "solvera" a **vstupných signálov** (ako vstup **$u(t)$** môžete zvoliť jednotkový skok, harmonickú funkciu, rýchlostnú funkciu, obdĺžnikový signál)
- vykreslite priebehy všetkých sledovaných veličín v case najprv** s využitím bloku „**oscilloscope**“ **v Simulinku** a **nasledne vykreslite priebehy sledovaných veličín aj v MATLABE**, pričom zabezpečíte vhodné rozmiestnenie fyzikálnych veličín do grafov s odpovedajúcim názvom, označením osí, legendou.
- Porovnajte **riešenie získané solverom ode45 v Matlabe** pri zvolenom LDS s riešením získaným v Simulinku. Výsledky zhody ilustrujte na grafických výstupoch

rlc45nab.m

```
t0=0; tfin=0.2;
x0=[0 0];
[t,x] = ode45('nabikond',[t0,tfin],x0);
uC=x(:,1); ic=x(:,2);
uR = ic*5; uL=(10-uR-uC);
subplot(231), plot(t,x(:,1)), title('uC v. t');
subplot(232), plot(t,x(:,2)), title('i v. t');
subplot(233), plot(uC,ic), title('I v. uC');
subplot(234), plot(t,uL), title('uL v. t');
subplot(235), plot(t,uR), title('uR v. t');
subplot(236), plot(uR,uL), title('uL v. uR');
```

nabikond.m

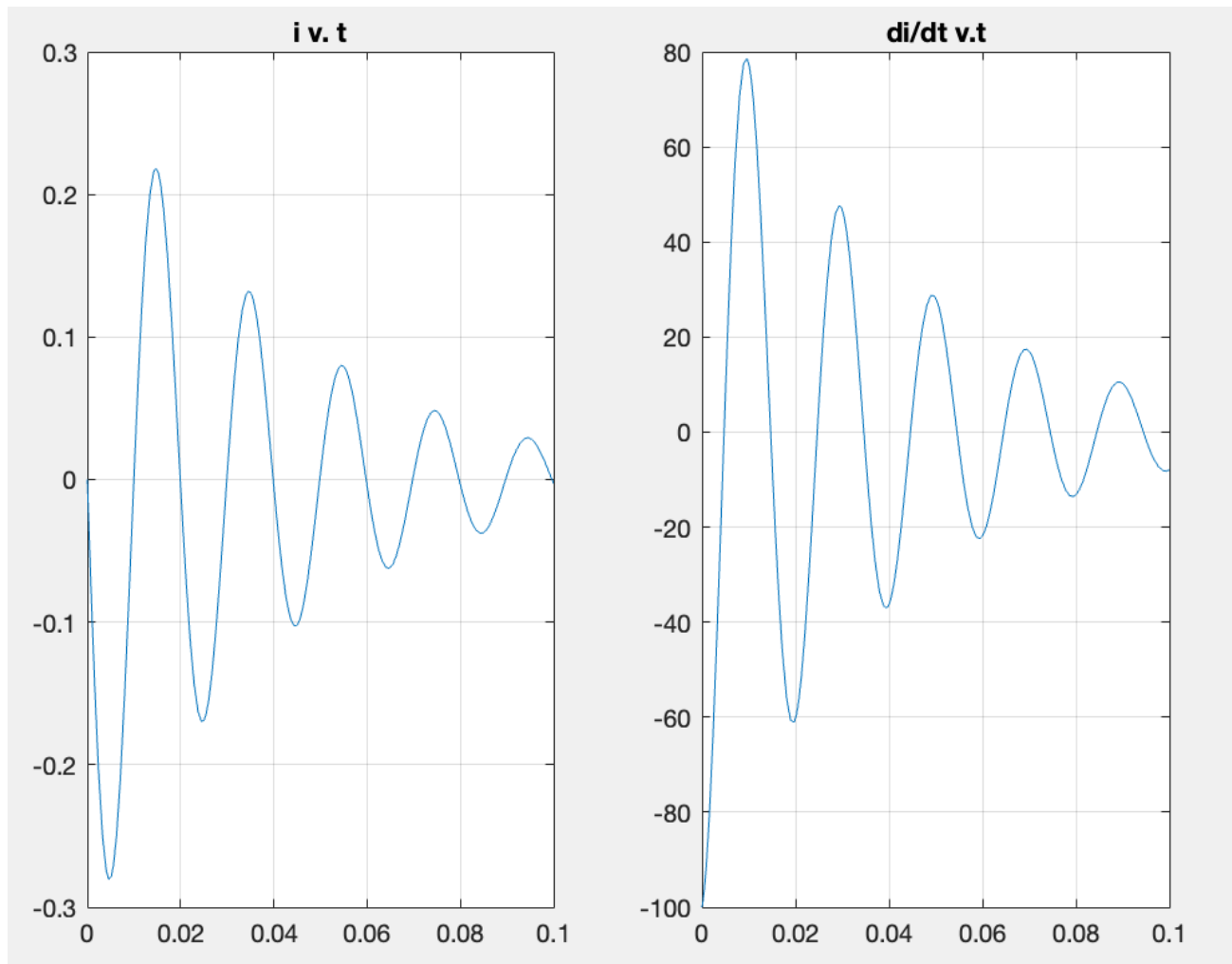
```
function xdot = nabikond(t,x)
R=5; L=0.1; C=1e-4; Udc=10;
xdot = [x(2)/C; 1/L*C*(Udc-R*x(2)-x(1))];
```

rlc23vyb.m

```
clc; clear;
t0=0; tfin=0.1;
x0=[0;-100];
tol=1e-3;
[t,x]=ode23('vybikond',[t0,tfin],x0)
ic=x(:,1);
di=x(:,2);
uR=ic*5;
uL=di*0.1;
uC=(-uR-uL);
subplot(121), plot(t,x(:,1)),grid, title('i v. t');
subplot(122), plot(t,x(:,2)),grid, title('di/dt v.t');
```

vybikond.m

```
function xdot=vybikond(t,x); % vracia stavové
R=5; L=0.1; C=1e-4;
xdot=[x(2);1/L*(-R*x(2)-(1/C)*x(1))];
```



Modelovanie/simulacia nelinearnych dynamickych systemov v prostredi Simulink

Uvazujte dynamicke systemy popisane **nelinearnymi diferencialnymi rovniciami** :

- **van der Polov oscilator** - https://en.wikipedia.org/wiki/Van_der_Pol_oscillator
- **matematicke kyvadlo** - https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-832-underactuated-robotics-spring-2009/readings/MIT6_832s09_read_ch02.pdf

Uloha 1 (T3_C). Z uvedenych **nelinearnych dynamickych systemov** analyzujte **DVA** NDS z pohladu:

1a) **vytvorenia simulacneho modelu Simulinku**, pri zvolenych NDS uvedte vstup/vystup a parametre NDS,

1b) **pouziva zapis matematickych modelov v tvare NDR**,

1c) modely NDS **zapiste** do **substitucneho kanonickeho tvaru**,

1d) **implementujte matematicky model NDS reprezentovany NDR do prostredia Simulink** a vykonajte simulacie s vyuzitim **zvoleneho solvera, kroku a casu simulacie**

1e) **vykreslite v MATLABE priebehy vsetkych sledovanych velicin v case**, **graficke vysledky riesenia NDR umiestnite do grafov s odpovedajucim nazvom, oznacenim osi, legendou a mriezkou.**

1f) **simulacne vysledky** ziskane riesenim NDR zo Simulinku **porovnajte s riesenim NDR v Matlabe** (pri rovnakych modeloch, rovnakych solveroch a case simulacie)

main.m

```
mu = 1;
tspan = [0 20];
x0 = [2; 0];
h = 0.1;

[t1, X1] = ode45(@(t, x) van_der_pol(t, x, mu), tspan, x0);
out = sim("model.slx");

hold on
plot(t1, X1(:,1), 'b', 'LineWidth', 1.5);
plot(out.simout.Time, out.simout.Data(:, 1), 'g*');
hold off

xlabel('Time');
ylabel('x_1');
title('Van der Pol Oscillator');
legend(["ode45" "Simulink"])
grid on;
```

van_der_pol.m

```
function dxdt = van_der_pol(t, x, mu)
    dxdt = zeros(2,1);
    dxdt(1) = x(2);
    dxdt(2) = mu * (1 - x(1)^2) * x(2) - x(1);
end
```

